Rabta Programming Language

Phase 2: ahead-of-time compiler to LLVM IR and native executables

Ansa Mubasher, Ahmed Yasin, Rana Muhammad Abdullah

Contents

Rabta Programming Language **1**

1. Dependencies and installation **2**

2. Building the compiler **3**

3. Compiling and running Rabta programs **3**
Examples . . . 4
Tests . . . 5

4. Language reference **6**
4.1 Lexical structure . . . 6
4.2 Types . . . 6
4.3 Program structure . . . 6
4.4 Functions . . . 7
4.5 Variables and arrays . . . 7
4.6 Expressions and precedence . . . 7
4.7 Control flow . . . 7
4.8 Output . . . 8
4.9 Protocols, channels and actors . . . 8

5. The semantic pass **8**
5.1 Type checking (semantic.c) . . . 9
5.2 Protocol checking (protocol.c) . . . 9
5.3 Other errors . . . 10

6. Implementation **10**
6.1 Code generation with the LLVM C API . . . 11
6.2 Actors and channels in IR . . . 12
6.3 Checks done . . . 12

7. Files **12**

8. Grammar summary **13**

Rabta Programming Language

Course: Compiler Construction (7th semester) **Authors:** Ansa Mubasher, Ahmed Yasin, Rana Muhammad Abdullah **Phase 2:** ahead-of-time (AOT) compiler `rabtac`: Flex + Bison front end, LLVM IR code generation with the LLVM C API, native executables built by `clang` (or `llc`)

Rabta is a small concurrent, message-passing language. A program is a set of independent actors (`banda`) that talk to each other only through typed channels (`nali`). The order, direction and type of every message on a channel is

written down as a conversation protocol (`baatcheet`), and the compiler checks **at compile time** that every actor follows it. Programs that would deadlock or exchange the wrong data are rejected before any code is generated.

In Phase 1 Rabta programs were executed by a tree-walking interpreter. In Phase 2 the same annotated AST is translated into an **LLVM IR module** (`.ll`), which `clang` compiles into a **native x86-64 executable**. Actors become POSIX threads and channels become mutex/condition-variable queues; the code for both is generated directly in the IR, so a compiled Rabta program needs nothing but libc and libpthread.

```
// Protocol: who speaks, in what order, with what type
baatcheet Hisaab shuru
    client bolega i64       // n
    server bolega i64       // sum of 1..n
khatam

banda server(sira s : Hisaab.server) shuru
    rakho n = suno s        // receive n
    rakho total = 0
    ghoom [i : 1 -> n + 1] chal
        total = total + i
    khatam
    bolo s, total           // send the answer; protocol now complete
khatam

kaam main() shuru
    nali line : Hisaab              // creates line.client and line.server
    paida server(line.server)       // spawn the server actor
    bolo line.client, 100           // send n
    rakho ans = suno line.client    // receive the answer
    dikhao(ans)                     // prints 5050
    waapis 0
khatam
```

```
$ ./rabtac examples/hisaab.rabta -o hisaab      # compile to a native binary
$ ./hisaab
5050
$ file hisaab
hisaab: ELF 64-bit LSB pie executable, x86-64, dynamically linked, ...
```

If `main` calls `suno line.client` before `bolo line.client, 100`, Rabta refuses to compile the program, and no IR or executable is produced:

```
$ ./rabtac tests/client_suno_first.rabta
tests/client_suno_first.rabta:22: error: protocol violation: Hisaab.client must
speak first (expected bolo i64, found suno)
1 error(s) found; program not compiled.
```

1. Dependencies and installation

Everything runs on Linux (tested on Ubuntu 24.04 under WSL 2).

Tool	Tested version	Purpose
gcc	13.3	builds the compiler itself (C11)
flex	2.6.4	lexer generator
bison	3.8.2	LALR(1) parser generator
llvm, llvm-dev	18.1	LLVM C API headers/library (`llvm-config`), `llc`, `llvm-as`
clang	18.1	compiles the generated `.ll` into a native executable

Tool	Tested version	Purpose
make	4.3	build
pandoc + pdflatex	3.1	only to regenerate README.pdf

Install everything needed to build and use the compiler:

```
sudo apt update
sudo apt install build-essential flex bison make llvm llvm-dev clang
```

Check the LLVM installation:

```
llvm-config --version        # 18.1.3
clang --version
```

If your distribution only installs versioned names (for example `llvm-config-18`, `clang-18`), pass them explicitly:

```
make LLVM_CONFIG=llvm-config-18
RABTA_CLANG=clang-18 RABTA_LLC=llc-18 ./rabtac program.rabta
```

Optional, only needed for `make pdf`:

```
sudo apt install pandoc texlive-latex-recommended texlive-latex-extra
```

2. Building the compiler

```
make            # builds ./rabtac
make examples   # compiles every examples/*.rabta into build/<name>
make ir         # writes the LLVM IR of every example to build/<name>.ll
make test       # builds, then runs the whole test suite (PASS/FAIL)
make clean      # removes every generated file (including build/)
make pdf        # regenerates README.pdf from README.md
```

`make` runs this pipeline:

```
bison -d -Wall rabta.y                    # -> rabta.tab.c, rabta.tab.h
flex rabta.l                              # -> lex.yy.c
gcc -std=gnu11 -Wall -Wextra -O2 -g $(llvm-config --cflags) -c ...
gcc ... -o rabtac *.o $(llvm-config --ldflags --libs core analysis target native)
```

The grammar has **0 shift/reduce and 0 reduce/reduce conflicts**, and all C files compile with **no warnings** under `-Wall -Wextra`.

3. Compiling and running Rabta programs

```
./rabtac program.rabta                # -> native executable ./program
./rabtac program.rabta -o app         # -> native executable ./app
./rabtac -S program.rabta             # -> LLVM IR only: ./program.ll
./rabtac -S program.rabta -o out.ll   # -> LLVM IR only: ./out.ll
./rabtac --keep-ll program.rabta      # executable + keep ./program.ll
./rabtac --llc program.rabta          # build through llc + link
./rabtac -O0 program.rabta            # optimisation level -O0 .. -O3 (default -O2)
./rabtac -v program.rabta             # show the clang / llc commands
./rabtac --run program.rabta          # compile to a temp dir, run, clean up
./rabtac --check program.rabta        # lexer + parser + semantic pass only
./rabtac --ast program.rabta          # also print the annotated AST
```

Without `-o`, the output is written to the current directory and named after the source file (`examples/hisaab.rabta -> ./hisaab`).

What happens inside `rabtac`:

```
program.rabta --Flex/Bison--> AST --semantic + protocol checks--> annotated AST
              --LLVM C API--> program.ll --clang -O2 ... -lpthread--> program
```

With `--llc` the last step is split into the two classic back-end steps:

```
llc -O2 -filetype=obj -relocation-model=pic program.ll -o program.o
clang program.o -o program -lpthread
```

The `.ll` file can also be compiled by hand with any LLVM 18 toolchain:

```
./rabtac -S examples/hisaab.rabta -o hisaab.ll
clang -O2 hisaab.ll -o hisaab -lpthread
./hisaab
```

Exit status of `rabtac`:

Status	Meaning
0	executable (or `.ll`) written; with `--run`: the program's own status
1	lexical, syntax, type or protocol error: **no IR and no executable**
64 / 66	bad command line / source file cannot be opened
70	IR generation or clang/llc failed (e.g. clang not installed)

Exit status of a **compiled program**:

Status	Meaning
value of `waapis` in `main` (mod 256)	the program ran (all examples return 0)
2	runtime error (division or modulo by zero, thali index out of bounds)

Examples

File	What it shows	Output
`examples/hisaab.rabta`	the proposal example: client asks a server actor for sum 1..n	`5050`
`examples/factorial.rabta`	recursion with `kaam`/`bulao`/`waapis`	0! .. 10!, 20!
`examples/bubble_sort.rabta`	bubble sort on a `thali` passed by reference	sorted array
`examples/fibonacci.rabta`	`jabtak` loop	first 20 Fibonacci numbers
`examples/if_else.rabta`	`agar` / `warna agar` / `warna`, logical operators, precedence	
`examples/ping_pong.rabta`	two actors playing ping-pong over one channel	
`examples/three_actors.rabta`	main + two worker actors summing ranges in parallel	`4501500`
`examples/pipeline.rabta`	a chain of three actors, each with two endpoints	`576`
`examples/sort_server.rabta`	sending a whole `thali` as a message	sorted array

```
$ make examples
./rabtac examples/ping_pong.rabta -o build/ping_pong
...
$ ./build/ping_pong
starting ping-pong
ping 1
    pong 1
ping 2
```

```
    pong 2
ping 3
    pong 3
pinger got the last pong: 3

$ ./rabtac tests/division_by_zero.rabta -o dz && ./dz; echo "exit $?"
tests/division_by_zero.rabta:6: runtime error: division by zero
exit 2
```

Tests

`make test` runs `tests/run_tests.sh`. Every program is compiled **ahead of time** into a native executable in `build/tests/`, and that executable is run. The suite has 63 checks in five groups:

1. **examples, clang pipeline:** every `examples/*.rabta` is compiled, must exit with 0 and print exactly `examples/expected/NAME.out`;
2. **examples, llc pipeline:** the same, built with `--llc`;
3. **code generation tests** (`tests/aot/`): programs that target the generated code, compared with `tests/aot/expected/NAME.out`. Their expected outputs were produced by the Phase 1 interpreter, so these tests also show that the compiler and the interpreter agree;
4. **LLVM IR checks:** `rabtac -S` must produce IR that `llvm-as` accepts, that defines `i32 @main`, and that declares `@pthread_create` whenever the program has a `banda`;
5. **invalid programs** (`tests/*.rabta`): the first line, `// EXPECT: <message>`, gives the text that must appear on stderr. Compile-time errors must make `rabtac` exit with 1, print nothing on stdout **and produce no executable**. Runtime-error tests declare `// EXPECT_EXIT: 2`; they compile, and the compiled program must fail.

Code generation test	What it checks
`aot/arith`	wrap-around, `INT64_MIN / -1`, signed `/` and `%`, comparisons, short-circuit `&&`/`||`
`aot/recursion`	mutual recursion with forward references, Ackermann, recursion 10,000 deep, 6 arguments
`aot/arrays`	heap `thali`, pass by reference, re-declaration inside a loop, a 1,000,000-element array
`aot/control`	`waapis` inside loops and branches, `warna agar` chains, nested loops, empty ranges
`aot/many_actors`	40 actor threads (thread registry grows), actors that spawn actors
`aot/queue`	20 messages in a row (more than the queue capacity of 16), `thali` both ways
`aot/exit_status`	`waapis 300` from `main` gives process exit status 44

Invalid program	Rejected because
`client_suno_first`	client receives before it sends (the proposal's example)
`server_bolo_first`	server sends before it receives
`wrong_message_type`	client sends `i64` where the protocol says `thali[3]`
`recv_wrong_type`	server receives an `i64` message into a `thali[4]`
`unfinished_conversation`	`main` returns before receiving the answer
`unfinished_actor`	actor ends before sending its answer
`use_after_finished`	endpoint used after all protocol steps are done
`use_after_handoff`	`main` uses an endpoint it gave to an actor with `paida`
`endpoint_never_used`	nobody ever plays the server role
`branch_mismatch`	only one branch of `agar`/`warna` sends
`protocol_in_loop`	conversation inside a loop
`wrong_endpoint_role`	actor needs `Hisaab.server`, got `Hisaab.client`
`type_mismatch`	`i64 + thali`
`undeclared_variable`	variable used before `rakho`
`wrong_arg_count`	`bulao` with too many arguments
`missing_return`	a `kaam` path without `waapis`
`syntax_error`	missing `)`

Invalid program	Rejected because
`division_by_zero`	runtime error in the compiled program (exit 2)
`modulo_by_zero`	runtime error in the compiled program (exit 2)
`index_out_of_bounds`	runtime error in the compiled program (exit 2)
`negative_index`	runtime error in the compiled program (exit 2)
`actor_runtime_error`	division by zero inside an actor thread ends the program (exit 2)

4. Language reference

4.1 Lexical structure

- **Comments** start with `//` and run to the end of the line.
- **Identifiers:** letters, digits and `_`, not starting with a digit.
- **Integers:** decimal literals that fit in a signed 64-bit integer. Negative numbers are written with unary minus (`-5`).
- **Strings:** `"text"`, with escapes `\n`, `\t`, `\\`, `\"`. They are only allowed as arguments of `dikhao`.
- Whitespace, including newlines, is not significant. Statements need no separator.

Keywords

Keyword	Meaning	Keyword	Meaning
`shuru` / `khatam`	begin / end	`baatcheet`	protocol declaration
`agar` / `warna`	if / else	`bolega`	protocol step: "X speaks T"
`jab_tak ... chal`	while loop	`banda`	actor declaration
`ghoom [...] chal`	range loop	`sira`	endpoint parameter type
`rakho`	declare a variable	`nali`	create a channel
`thali`	bounded array	`paida`	spawn an actor
`kaam`	function definition	`bolo`	send a message
`bulao`	function call	`suno`	receive a message
`waapis`	return	`dikhao`	print
`i64`	64-bit integer type		

`client` and `server` are **not** keywords. They are role names that are checked by the semantic pass, so an actor may be called `server`, as in the proposal.

4.2 Types

Type	Description
`i64`	signed 64-bit integer. Booleans are `i64`: 0 is false, anything else is true
`thali[N]`	bounded array of N `i64` values (1 <= N <= 10,000,000), zero-initialised
channel	created by `nali ch : P`; it has the two endpoints `ch.client` and `ch.server`
endpoint	`sira e : P.client` or `sira e : P.server` (actor parameters only)

Channels and endpoints are not values. They cannot be stored in variables, compared or printed. They can only be used with `bolo`, `suno` and `paida`. This keeps every endpoint statically known, which the protocol checker needs.

4.3 Program structure

A program is a list of protocol, actor and function declarations in any order. Declarations may refer to each other (recursion and forward references are allowed). There are **no global variables**. Execution starts at `kaam main()`, which takes no parameters and must return an `i64` with `waapis`.

4.4 Functions

```
kaam name(param, param, ...) shuru
    statements
khatam
```

- A parameter is written `x` or `x : i64` (an `i64` passed by value), or `thali a` (an array passed **by reference**, of any size).
- A function always returns an `i64`. **Every path must end in `waapis expr`**: a function whose end is reachable is a compile-time error.
- Calls use `bulao`, either as an expression or as a statement:

```
rakho f = bulao factorial(10)
bulao sort(nums, 8)                 // result discarded
dikhao(bulao add(2, 3))
```

4.5 Variables and arrays

```
rakho x = 5 * 2                     // declare an i64 (initialiser required)
x = x + 1                           // assign

thali a[5]                          // 5 zeros
thali b[5] = [3, 1, 4]              // [3, 1, 4, 0, 0]  (at most N values)
b[0] = b[1] + 10                    // index with [expr], 0-based
```

- Indices are checked at runtime. An out-of-range index is a runtime error (exit 2).
- A whole `thali` cannot be assigned or used in arithmetic. It can be printed, passed to a `kaam` by reference, sent as a message, or received into.
- Names are block-scoped. Each `shuru ... khatam` body, loop body and branch is a new scope. Redeclaring a name in the same scope is an error, and shadowing an outer name is allowed.

4.6 Expressions and precedence

From lowest to highest precedence:

Level	Operators	Associativity
1	`||`	left
2	`&&`	left
3	`== !=`	non-associative
4	`< <= > >=`	non-associative
5	`+ -`	left
6	`* / %`	left
7	unary `-`, `!`	prefix

Operands are evaluated left to right. `&&` and `||` short-circuit and give 0 or 1, and comparisons give 0 or 1. Arithmetic wraps around on overflow (two's complement). `/` and `%` truncate toward zero, and dividing by zero is a runtime error.

4.7 Control flow

```
agar x > 0 shuru                    // if
    ...
khatam warna agar x == 0 shuru      // else if
    ...
khatam warna shuru                  // else
    ...
khatam
```

```
jab_tak i < 10 chal                 // while
    ...
khatam

ghoom [i : a -> b] chal             // for i = a, a+1, ..., b-1
    ...
khatam
```

The range loop evaluates `a` and `b` once. `b` is **exclusive**, and `i` is a new `i64` variable that is local to the loop.

4.8 Output

```
dikhao(expr_or_string, ...)
```

`dikhao` prints its arguments separated by one space, followed by a newline. `i64` values are printed in decimal, a `thali` as `[1, 2, 3]` and strings as they are. Each `dikhao` line is printed atomically, so lines from different actors never mix.

4.9 Protocols, channels and actors

Protocol. A protocol is a fixed, non-empty sequence of steps. Each step says which role speaks and the type of the message (`i64` or `thali[N]`):

```
baatcheet Tartib shuru
    client bolega thali[6]
    server bolega thali[6]
khatam
```

Channel. `nali ch : P` creates a channel for protocol `P` with the two endpoints `ch.client` and `ch.server`.

Actor. An actor is declared like a function, but with `banda`. It has no return value and may not contain `waapis`. Its parameters may be `i64` values or endpoints:

```
banda worker(id, sira job : Hissa.server) shuru ... khatam
```

Actors cannot take `thali` parameters, because actors share nothing. Send the array as a message instead.

Spawning. `paida name(args)` starts the actor on a new thread. `i64` arguments are copied. Endpoint arguments are **handed over**: the spawner may not use that endpoint again. An actor may pass its own endpoint on to another actor in the same way.

Messaging.

```
bolo ep, expr                       // send an i64 or a whole thali
rakho x = suno ep                   // receive an i64 into a new variable
x = suno ep                         // ... into an existing variable
a[i] = suno ep                      // ... into an array element
thali t[6] = suno ep                // receive a thali[6] into a new array
suno ep                             // receive and discard
```

Here `ep` is `ch.client`, `ch.server` or a `sira` parameter. `suno` is only allowed as a whole initialiser or assigned value, or as a statement. That way the textual order of statements is exactly the order of protocol actions. Messages are copied (a `thali` is deep-copied), so no memory is ever shared.

Program end. When `main` returns, the compiled program waits for every actor, including actors started by other actors, to finish. Then it exits with `main`'s return value.

5. The semantic pass

The semantic pass runs after parsing and before any code is generated. If it reports any error, **no LLVM IR and no executable are produced** and `rabtac` exits with status 1. Each error has the form `file:line: error: message`.

5.1 Type checking (semantic.c)

- undeclared variables, channels, endpoints, functions, actors and protocols;
- duplicate declarations (top level, and within one scope);
- type mismatches in operators, conditions, indices, assignments, `rakho`, `waapis`, call arguments, `dikhao` and `bolo`;
- argument counts and kinds for `bulao` and `paida`, including the endpoint protocol and role (`expected sira Hisaab.server, found sira Hisaab.client`);
- return rules: every `kaam` path must end in `waapis` with an `i64`, `waapis` is not allowed in a `banda`, `main` must exist, take no parameters and not be called;
- `sira` parameters only on actors, and `thali` parameters only on functions;
- protocol roles must be `client`/`server`, and array sizes must be in range.

The pass also annotates the AST. Every variable gets a frame slot, every function and actor gets its frame size, and every name points to its declaration. The LLVM code generator uses these annotations directly: slots become `alloca`s, resolved declarations become direct `call`s, and endpoint roles become constant queue indices.

5.2 Protocol checking (protocol.c)

Each endpoint is a small state machine whose state is "how many protocol steps are done". The checker walks the body of `main`, of every `kaam` and of every `banda` in program order and simulates these state machines statically. One endpoint is tracked per `ch.client`/`ch.server` created by `nali`, and one per `sira` parameter.

Situation	Error
`suno` when this role must speak	`Hisaab.client must speak first (expected bolo i64, found suno)`
`bolo` when this role must listen	`Hisaab.server must listen first (expected suno i64, found bolo)`
(later steps)	`... must speak at step 3 (...) / ... must listen at step 2 (...)`
wrong type sent	`Tartib.client sends the wrong message type at step 1 (expected bolo thali[3], found bolo i64)`
wrong type received	`Hisaab.server receives the wrong message type at step 1 (expected suno i64, found suno into thali[4])`
use after the last step	`Hisaab.client is used after the conversation finished (...)`
unfinished at `waapis` / end of scope	`conversation on line.client (Hisaab.client) is unfinished at waapis (next expected: suno i64, step 2 of 2)`
endpoint never used	`endpoint line.server (Hisaab.server) is never used in main`
used after `paida` handed it away	`line.server (Hisaab.server) was handed to banda 'server' with paida on line 16 and cannot be used here any more`
handed away after being used	`cannot hand line.client (Hisaab.client) to banda 'x' after it has been used here`
branches disagree	`the branches of agar leave line.client (Hisaab.client) in different states (agar: 1 step(s) done, warna: 0 step(s) done)`

Situation	Error
`agar` without `warna` talks	`agar without warna changes the state of ... ; add a warna branch that does the same conversation`
conversation inside a loop	`line.client (Hisaab.client) is used inside the ghoom loop on line 17; loop bodies must be protocol-neutral`

Branches (`agar`/`warna`). Both branches start from the same endpoint states and must end in the same states. An `agar` without `warna` must not change any endpoint. A branch that ends in `waapis` does not have to match, because its endpoints were already checked at that `waapis`.

Loops: chosen rule. A protocol has a fixed number of steps, but a loop may run any number of times. So a `jab_tak`/`ghoom` body must be **protocol-neutral**: it may not use, or hand to `paida`, any endpoint declared outside the loop. A channel created *inside* the loop body is allowed if its conversation finishes within the same iteration:

```
ghoom [i : 1 -> 4] chal
    nali c : Q                      // fresh channel each iteration: OK
    paida doubler(c.server)
    bolo c.client, i
    rakho r = suno c.client
khatam
```

What this guarantees. Every endpoint performs exactly the steps of its protocol, in order, with the right types, and one side always has the opposite view of the other. So two actors talking over one channel can never wait for each other forever, and can never receive data of an unexpected type.

Limitation. As with basic session types, a cycle of waits across several different channels (A waits for B on channel 1 while B waits for A on channel 2) is not detected.

5.3 Other errors

```
file:4: syntax error, unexpected dikhao                 (parser)
file:3: error: unexpected character '@'                 (lexer)
file:6: runtime error: division by zero                 (exit 2)
file:6: runtime error: index 5 is out of bounds for thali 'a' of size 5
```

6. Implementation

```
 .rabta source
      |
  rabta.l  (Flex)      tokens + line numbers (yylineno -> Bison locations)
      |
  rabta.y  (Bison)     LALR(1) grammar, actions build the AST (ast.c)
      |                the start rule's action drives the steps below
  semantic.c           symbol tables, type checking, slot allocation
  protocol.c           protocol state-machine simulation
      |                any error -> report, exit 1, no IR is generated
  llvm_gen.c           LLVM context/module/builder, types, variables, dikhao
  llvm_flow.c          arithmetic, comparisons, agar/warna, loops
  llvm_func.c          kaam, bulao, waapis, the C main()
  llvm_actor.c         banda threads, nali queues, bolo/suno (pthreads)
      |                LLVMVerifyModule, then LLVMPrintModuleToFile
  program.ll
      |
  toolchain.c          clang -O2 program.ll -lpthread   (or llc + link)
      |
```

native executable

6.1 Code generation with the LLVM C API

`llvm_generate()` creates an `LLVMContextRef`, an `LLVMModuleRef` (with the host's target triple and data layout) and an `LLVMBuilderRef`, walks the annotated AST, checks the result with `LLVMVerifyModule` and writes it with `LLVMPrintModuleToFile`.

Type mapping.

Rabta	LLVM IR
`i64`	`i64`
condition	`i1` (`icmp ne %x, 0`), widened back to `i64` with `zext`
`thali[N]`	heap block of `i64` (`ptr`) plus its length (`i64`)
`nali` / `sira` endpoint	`ptr` to a `%rabta.chan`; the endpoint's role is known at compile time
`kaam f(a, thali t)`	`define internal i64 @kaam.f(i64 %a, ptr %t, i64 %t.len)`
`banda w(...)`	`define internal ptr @banda.w(ptr %args)` (a pthread start routine)

Variables. Every variable has a frame slot number from the semantic pass. Each slot is an `alloca` in the function's entry block (`%total = alloca i64`); `rakho` and assignment are `store`s and uses are `load`s. clang's `mem2reg` pass turns them into SSA registers at `-O1` and above.

Arithmetic. `+ - *` are `add`/`sub`/`mul` without the `nsw` flag, so they wrap around like in Phase 1. `/` and `%` are `sdiv`/`srem` behind two checks: a divisor of 0 branches to a runtime error, and a divisor of -1 is replaced by `0 - x` (or 0), because `INT64_MIN / -1` traps on x86-64. Comparisons are `icmp slt/sle/sgt/sge/eq/ne`.

Control flow. Every construct gets its own basic blocks:

```
agar c ... warna ...     br i1 %c, label %if.then, label %if.else  -> %if.end
jab_tak c chal ...       %while.cond -> %while.body -> %while.cond / %while.end
ghoom [i : a -> b] ...   %for.cond -> %for.body -> %for.inc -> %for.cond / %for.end
a && b, a || b           %and.rhs / %and.end with a phi (short-circuit)
```

Code that follows a `waapis` is put in a block of its own, which LLVM then removes as unreachable.

Functions. All `kaam`s are declared first, so calls may come before definitions (and mutual recursion works). `bulao` is a `call i64`, `waapis` is `ret i64`. A `thali` argument is passed by reference as its element pointer and length. Before every `ret` the function `free`s the arrays it declared. The names get a `kaam.` / `banda.` prefix, so a Rabta `kaam printf` cannot clash with libc. The real C entry point is generated as:

```
define i32 @main(i32 %argc, ptr %argv) {
  call void @rabta.init()
  %rv = call i64 @kaam.main()
  call void @rabta.join_all()          ; wait for every banda
  call void @rabta.shutdown()          ; free channels
  ...
  ret i32 %exit.code                    ; %rv & 255
}
```

Arrays. `thali a[N]` is `calloc(N, 8)` (zero-filled). If the declaration runs again in a loop, the old array is freed first. Every `a[i]` checks `i <u N` (one unsigned compare catches negative indices too) and branches to `@rabta.index_fail` when it fails.

Output. `dikhao` formats its line into a private buffer (`open_memstream`), in argument order, then `@rabta.emit` writes the whole line to stdout under `flockfile`, so lines from different actors never mix.

Runtime errors. `@rabta.fail` / `@rabta.index_fail` take the stdout lock, flush stdout, print `file:line: runtime error: ...` to stderr and `exit(2)`, which also stops every actor thread. The source file name is a string constant in the module.

6.2 Actors and channels in IR

The actor runtime is not a separate C library. It is generated into every module that needs it, as IR functions that call declared POSIX functions:

```
declare i32 @pthread_create(ptr, ptr, ptr, ptr)
declare i32 @pthread_join(i64, ptr)
declare i32 @pthread_mutex_lock(ptr)
declare i32 @pthread_mutex_unlock(ptr)
declare i32 @pthread_cond_wait(ptr, ptr)
declare i32 @pthread_cond_signal(ptr)

%rabta.msg   = type { i64, ptr }                ; value or length, thali copy
%rabta.queue = type { [16 x %rabta.msg], i64, i64,   ; ring buffer, head, count
                      [8 x i64], [8 x i64], [8 x i64] } ; mutex, not_empty, not_full
%rabta.chan  = type { [2 x %rabta.queue], ptr }  ; one queue per receiving role
```

IR function	Used for	What it does
`@rabta.chan_new`	`nali`	`calloc` a channel, `pthread_mutex_init` / `pthread_cond_init` both queues, add it to the channel list
`@rabta.send`	`bolo`	lock, `pthread_cond_wait(not_full)` while 16 messages are queued, enqueue, signal `not_empty`, unlock
`@rabta.recv`	`suno`	lock, `pthread_cond_wait(not_empty)` while empty, dequeue, signal `not_full`, unlock
`@rabta.spawn`	`paida`	`pthread_create` with a 16 MiB stack, record the thread id in a growing registry
`@rabta.join_all`	end of `main`	`pthread_join` every registered thread, including ones started while joining
`@rabta.shutdown`	end of `main`	destroy and free every channel and the registry

`paida w(x, ch.server)` evaluates its arguments, packs them into a heap struct (`{ i64, ptr }` here) and passes it to `@rabta.spawn(@banda.w, %args)`. The actor unpacks its parameters and frees the struct. `bolo` of a `thali` sends a `malloc`ed copy; `suno` copies it into the receiver's array and frees the message, so actors never share memory.

6.3 Checks done

- `make test`: **63 passed, 0 failed** (9 examples with clang, 9 with llc, 7 code generation tests, 16 IR checks, 22 invalid programs).
- The examples and code generation tests also give identical output when compiled with `-O0`.
- The concurrent programs (`ping_pong`, `aot/queue`, `aot/many_actors`) were each run 300 times with identical output.
- The generated IR of every example and code generation test was compiled with `clang -fsanitize=address` (no memory errors, no leaks) and with `clang -fsanitize=thread` (no data races).

7. Files

File	Description
rabta.l	Flex lexer: keywords, identifiers, integers, strings, operators, // comments, line tracking
rabta.y	Bison grammar (0 conflicts) with AST-building actions and error reporting; also the rabtac driver: main(), command-line options, and the pipeline semantic check -> IR -> executable
ast.h, ast.c	AST node, list and type definitions, constructors, --ast pretty printer, cleanup
semantic.h, semantic.c	scoped symbol table, type checking, call/return rules, slot allocation, error reporting
protocol.h, protocol.c	protocol checker: endpoint state machines, branches, loops, hand-over
llvm_gen.h	public interface of the code generator: llvm_generate(program, source, out.ll)
llvm_gen_priv.h	code generator state (Gen: context, module, builders, LLVM types, frame slots) shared by the llvm_*.c files
llvm_gen.c	LLVM context/module/builder setup, target data layout, type mapping, libc declarations, frame-slot allocas, rakho/assignment, thali allocation and bounds-checked indexing, dikhao, runtime error helpers, statement/expression dispatch, verification and output of the module
llvm_flow.c	arithmetic (with the division checks), comparisons, !, short-circuit &&/||, agar/warna and loop basic blocks
llvm_func.c	kaam declarations and definitions, parameter passing, bulao calls, waapis, the C main entry point
llvm_actor.c	channel and thread runtime generated in IR (pthread mutexes, condition variables, pthread_create/pthread_join), banda functions, paida, nali, bolo, suno
toolchain.h, toolchain.c	runs clang (or llc + clang) on the .ll file, and runs programs for --run
Makefile	all, examples, ir, test, clean, pdf targets; finds LLVM with llvm-config
examples/*.rabta	valid programs; examples/expected/*.out holds their expected output
tests/aot/*.rabta	Phase 2 code generation tests; tests/aot/expected/*.out holds their expected output
tests/*.rabta	programs that must be rejected (at compile time or at run time), each with its // EXPECT: message
tests/run_tests.sh	test runner used by make test (prints PASS/FAIL)
docs/Rabta_Compiler_Proposal.pdf	the original project proposal
README.md, README.pdf	this document
.gitignore	ignores generated files (lex.yy.c, *.tab.*, *.o, *.ll, rabtac, build/)

Generated files (never committed): rabtac, *.o, rabta.tab.c, rabta.tab.h, lex.yy.c, and everything in build/ (compiled programs and .ll files).

8. Grammar summary

```
program     : { protocol | actor | function }
protocol    : "baatcheet" ID "shuru" step { step } "khatam"
step        : ID "bolega" ( "i64" | "thali" "[" INT "]" )
actor       : "banda" ID "(" [ params ] ")" block
function    : "kaam"  ID "(" [ params ] ")" block
params      : param { "," param }
param       : ID [ ":" "i64" ] | "thali" ID | "sira" ID ":" ID "." ID
block       : "shuru" { stmt } "khatam"

stmt        : "rakho" ID "=" ( expr | recv )
            | "thali" ID "[" INT "]" [ "=" ( "[" [ args ] "]" | recv ) ]
            | ID "=" ( expr | recv )
```

```
            | ID "[" expr "]" "=" ( expr | recv )
            | "agar" expr block [ "warna" ( block | "agar" expr block ... ) ]
            | "jab_tak" expr "chal" { stmt } "khatam"
            | "ghoom" "[" ID ":" expr "->" expr "]" "chal" { stmt } "khatam"
            | "waapis" expr
            | "dikhao" "(" ( expr | STRING ) { "," ( expr | STRING ) } ")"
            | "bulao" ID "(" [ args ] ")"
            | "nali" ID ":" ID
            | "paida" ID "(" [ args ] ")"
            | "bolo" endpoint "," expr
            | recv
recv        : "suno" endpoint
endpoint    : ID | ID "." ID
expr        : expr binop expr | "-" expr | "!" expr | primary
primary     : INT | ID | ID "[" expr "]" | ID "." ID | "(" expr ")"
            | "bulao" ID "(" [ args ] ")"
args        : expr { "," expr }
```